

Dossier technique complet — Déploiement Pi-CloudDOOM

Objectif du document

Ce document sert de **preuve technique** pour la phase de déploiement du projet **Pi-CloudDOOM**. Il regroupe les technologies utilisées, les dépôts, les images Docker, les commandes, les fichiers Kubernetes, les étapes OpenStack, les corrections réalisées, les problèmes rencontrés et les procédures de vérification.

Ce fichier doit permettre à un membre de l'équipe ou à un jury de comprendre **comment le déploiement a été réalisé concrètement**, pas seulement théoriquement.

Il complète le scénario oral de validation. Le scénario explique comment parler au jury. Ce dossier montre les preuves techniques.

1. Vue globale du déploiement

Le projet Pi-CloudDOOM a été déployé selon une approche cloud-native basée sur :

- une application frontend Angular ;
- plusieurs microservices Spring Boot ;
- une authentification centralisée avec Keycloak ;
- une base de données PostgreSQL ;
- Redis pour le cache ;
- Kafka pour la communication asynchrone ;
- Docker pour la containerisation ;
- Docker Hub pour le registre d'images ;
- Kubernetes pour l'orchestration ;
- OpenStack pour l'infrastructure IaaS ;
- Cloudflare pour l'exposition publique ;
- Ollama / Azure pour la partie IA ;
- monitoring et détection d'anomalies pour l'exploitation.

L'objectif était de passer d'un environnement local de développement vers un environnement cloud capable d'exécuter les services de manière structurée, isolée, observable et défendable devant le jury.

2. Dépôts Git et organisation du code

2.1 Dépôt principal

Le dépôt principal du projet est :

```
https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM
```

2.2 Branches importantes utilisées

Pendant le projet, plusieurs branches ont été utilisées pour intégrer les travaux des membres :

```
main  
develop  
integration/final  
member-user-service  
feature/m2-interview  
integrate/m4-training  
integrate/m4-training-v2  
member-3-security-config  
member-5-kafka-events
```

La branche d'intégration finale utilisée pour regrouper les travaux était :

```
integration/final
```

2.3 Structure logique du projet

La structure du projet suit une logique microservices :

```
Pi-CloudDOOM/  
├─ frontend/           # Application Angular  
├─ user-service/       # Gestion utilisateurs, CV, profil, IA  
├─ community-service/  # Module communauté  
├─ interview-service/  # Module entretien/interview  
├─ training-service/   # Module formation  
├─ docker-compose.yml  # Orchestration locale  
├─ k8s/                # Fichiers Kubernetes si présents
```

```
|— README.md
|— autres services/modules
```

Remarque : les noms exacts des dossiers peuvent varier selon l'organisation finale du dépôt, mais la logique reste la même : chaque module applicatif est isolé et possède sa propre configuration.

3. Technologies utilisées

3.1 Backend

Technologie principale :

Spring Boot

Utilisation :

- création d'APIs REST ;
- logique métier ;
- sécurité via Spring Security ;
- validation des JWT Keycloak ;
- communication avec PostgreSQL ;
- intégration avec Redis, Kafka et service IA ;
- upload et parsing de CV.

Composants Spring importants :

Spring Web
Spring Security
Spring OAuth2 Resource Server
Spring Data JPA
Spring Boot Actuator

3.2 Frontend

Technologie :

Angular

Utilisation :

- interface utilisateur ;
- login ;
- profil utilisateur ;
- upload CV ;
- consultation CV ;
- appels REST vers les microservices ;
- intégration OAuth via Keycloak.

3.3 Authentification

Technologie :

Keycloak

Utilisation :

- serveur d'identité ;
- gestion des utilisateurs ;
- gestion des rôles ;
- génération des tokens JWT ;
- fournisseurs OAuth : Google, LinkedIn, GitHub ;
- intégration avec Angular et Spring Boot.

3.4 Données et middleware

Composants :

PostgreSQL
Redis
Kafka

Rôles :

- PostgreSQL : stockage relationnel ;
- Redis : cache / stockage rapide ;
- Kafka : communication événementielle entre microservices.

3.5 Containerisation et orchestration

Technologies :

Docker
Docker Hub
Kubernetes

Rôles :

- Docker : création des images ;
- Docker Hub : registre d'images ;
- Kubernetes : orchestration des conteneurs.

3.6 Infrastructure

Technologie :

OpenStack

Rôle :

- création des instances ;
- réseau ;
- volumes ;
- snapshots ;
- hébergement du cluster Kubernetes.

3.7 Exposition publique

Technologie :

Cloudflare

Rôle :

- DNS ;
- domaine public ;
- proxy ;
- HTTPS ;
- diagnostic des erreurs d'accès.

3.8 Intelligence artificielle

Technologies étudiées/utilisées :

Gemini
Ollama
Azure Container Apps

Rôle :

- extraction intelligente d'informations depuis un CV PDF ;
- enrichissement automatique du profil utilisateur ;
- séparation possible du service IA sur Azure pour réduire la charge du cluster OpenStack.

4. Architecture de déploiement

4.1 Architecture logique

L'architecture logique peut être présentée comme suit :

```
Utilisateur
|
v
Cloudflare / Domaine public
|
v
Frontend Angular
|
v
Microservices Spring Boot
|   |   |
v   v   v
PostgreSQL Redis Kafka
|
v
Keycloak pour authentification
|
v
Ollama / Azure AI pour parsing CV
```

4.2 Architecture Kubernetes

Dans Kubernetes, les composants sont organisés dans le namespace :

piclouddoom

Objets Kubernetes principaux :

Namespace
Deployments
Pods
Services
ConfigMaps
Secrets
PersistentVolumeClaims
StorageClass
Ingress ou exposition externe selon configuration

4.3 Justification de l'architecture

Nous avons choisi cette architecture pour les raisons suivantes :

- l'application est composée de plusieurs microservices ;
- chaque service doit pouvoir être déployé séparément ;
- Kubernetes permet la gestion des pods et la communication interne ;
- Docker standardise l'environnement d'exécution ;
- OpenStack permet de travailler sur une vraie infrastructure cloud privée ;
- Cloudflare permet de rendre l'application accessible publiquement ;
- Keycloak centralise la sécurité ;
- une stratégie hybride permet d'externaliser les composants lourds comme l'IA.

5. Containerisation avec Docker

5.1 Objectif

Chaque service doit être transformé en image Docker afin de pouvoir être lancé dans Kubernetes.

Docker résout les problèmes suivants :

- différences entre machines des développeurs ;
- dépendances locales ;
- versions Java/Node ;
- portabilité ;
- reproductibilité ;
- préparation à Kubernetes.

5.2 Exemple de Dockerfile pour un microservice Spring Boot

Exemple générique :

```
FROM eclipse-temurin:17-jre

WORKDIR /app

COPY target/*.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Version multi-stage possible :

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

5.3 Exemple de Dockerfile pour Angular

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist/ /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

5.4 Construction d'une image

Exemple :

```
cd user-service
mvn clean package -DskipTests

docker build -t docker.io/azizbna/pi-clouddoom-user-service:latest .
```

5.5 Push vers Docker Hub

```
docker login

docker push docker.io/azizbna/pi-clouddoom-user-service:latest
```

5.6 Image user-service utilisée pendant le déploiement

Une image utilisée pendant les corrections était :

```
docker.io/azizbna/pi-clouddoom-user-service:cv-ollama-fast-v2-v2
```

5.7 Vérifier qu'une image existe localement

```
docker images | grep pi-clouddoom
```

5.8 Tester un container localement

```
docker run --rm -p 8080:8080 docker.io/azizbna/pi-clouddoom-user-service:latest
```

5.9 Pourquoi taguer les images ?

Les tags permettent de savoir quelle version est déployée.

Exemples :

```
latest
v1
cv-ollama-fast-v2-v2
```

```
fix-google-oauth
stable-demo
```

Pour une validation, il est préférable d'utiliser un tag clair comme :

```
validation-phase5
```

ou :

```
demo-final
```

Cela évite d'utiliser `latest`, qui peut être ambigu.

6. Docker Compose local

6.1 Rôle de Docker Compose

Avant Kubernetes, Docker Compose permet de lancer plusieurs services localement :

- backend ;
- frontend ;
- Keycloak ;
- PostgreSQL ;
- Redis ;
- Kafka ;
- Ollama.

Cela permet de vérifier l'intégration avant le déploiement cloud.

6.2 Exemple de composants dans docker-compose

```
services:
  postgres:
    image: postgres:16-alpine

  redis:
    image: redis:7-alpine

  keycloak:
    image: quay.io/keycloak/keycloak:24.0.1
```

```
kafka:
  image: confluentinc/cp-kafka:7.5.0

ollama:
  image: ollama/ollama

user-service:
  image: docker.io/azizbna/pi-clouddoom-user-service:latest
```

6.3 Commandes Docker Compose

```
docker compose up -d

docker compose ps

docker compose logs -f user-service

docker compose down
```

6.4 Vérification des services locaux

```
docker ps
curl http://localhost:8080/actuator/health
curl http://localhost:11434/api/tags
```

7. Infrastructure OpenStack

7.1 Objectif d'OpenStack

OpenStack a été utilisé comme cloud privé pour créer les machines virtuelles du cluster Kubernetes.

Ressources gérées :

- instances ;
- volumes ;
- snapshots ;
- réseau ;
- adresses IP ;
- accès SSH ;
- stacks Heat.

7.2 Charger les credentials OpenStack

Avant d'utiliser les commandes OpenStack :

```
source ~/admin-openrc
```

ou selon l'environnement :

```
source ~/openrc
```

7.3 Lister les instances

```
openstack server list
```

7.4 Afficher une instance

```
openstack server show <server-name-or-id>
```

7.5 Lister les volumes

```
openstack volume list
```

7.6 Afficher un volume

```
openstack volume show <volume-id>
```

Exemple utilisé :

```
VOL=00404d60-d3e2-478d-81b0-aa7ff34f763d  
openstack volume show "$VOL" -c status -c attachments  
openstack volume snapshot list --volume "$VOL"
```

7.7 Supprimer un snapshot puis un volume

Si un volume possède un snapshot, supprimer d'abord le snapshot :

```
openstack volume snapshot delete <snapshot-id>
openstack volume delete <volume-id>
```

7.8 Lister les hyperviseurs

```
openstack hypervisor list
```

7.9 Vérifier les informations d'un hyperviseur

```
openstack hypervisor show <hypervisor-name>
```

Remarque : selon la version OpenStack, certaines colonnes comme `vcpus_used`, `memory_mb_used` ou `running_vms` peuvent ne pas être disponibles directement dans `openstack hypervisor show`. Dans ce cas, il faut utiliser d'autres commandes, l'API Nova ou consulter les informations disponibles dans la sortie complète.

7.10 Gestion des stacks Heat

Commandes utiles :

```
openstack stack list
openstack stack show <stack-name>
openstack stack create -t <template.yaml> <stack-name>
openstack stack update -t <template.yaml> <stack-name>
openstack stack delete <stack-name>
```

7.11 Problème SSH host key changed

Erreur rencontrée :

```
WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!
```

Cause : l'adresse IP a été réutilisée par une nouvelle instance mais l'ancienne clé SSH est encore dans `known_hosts`.

Correction :

```
ssh-keygen -f ~/.ssh/known_hosts -R <IP>
```

Puis reconnexion :

```
ssh -i ~/heat-eval-key.pem ubuntu@<IP>
```

8. Création et validation du cluster Kubernetes

8.1 Nœuds du cluster

Le cluster contenait un control plane et plusieurs workers.

Exemples de noms observés :

```
k8s-cp1
k8s-w1
k8s-w2
k8s-w3
k8s-w4
```

8.2 Vérifier les nœuds

```
kubectl get nodes
kubectl get nodes -o wide
```

Résultat attendu :

NAME	STATUS	ROLES	AGE	VERSION
k8s-cp1	Ready	control-plane	...	...
k8s-w1	Ready	<none>	...	...
k8s-w2	Ready	<none>	...	...
k8s-w3	Ready	<none>	...	...
k8s-w4	Ready	<none>	...	...

8.3 Vérifier les pods système

```
kubectl get pods -A
```

Éléments importants :

```
kube-system  
calico-system ou pods CNI  
local-path-storage
```

8.4 CNI réseau

Un CNI comme Calico est nécessaire pour permettre la communication entre pods.

Vérification :

```
kubectl get pods -A | grep -i calico
```

ou :

```
kubectl get pods -n kube-system
```

8.5 Namespace du projet

Créer le namespace si nécessaire :

```
kubectl create namespace piclouddoom
```

Vérifier :

```
kubectl get ns
```

9. StorageClass et persistance

9.1 Problème rencontré

Au début, aucune StorageClass n'était disponible :

```
kubectl get storageclass
```

Résultat :

No resources found

9.2 Installation de local-path-provisioner

Commande utilisée :

```
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/master/deploy/local-path-storage.yaml
```

Vérification :

```
kubectl get storageclass
```

Résultat attendu :

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE
local-path	rancher.io/local-path	Delete	WaitForFirstConsumer

9.3 Pourquoi c'est important ?

Sans StorageClass, les composants qui ont besoin de persistance ne peuvent pas créer leurs volumes automatiquement.

Composants concernés :

- PostgreSQL ;
- Keycloak si base persistante ;
- éventuellement Kafka ;
- éventuellement stockage de fichiers CV ;
- éventuellement Ollama si le modèle doit être conservé.

10. Fichiers Kubernetes principaux

10.1 Namespace

```
apiVersion: v1  
kind: Namespace
```

```
metadata:
  name: piclouddoom
```

10.2 Exemple de Deployment Spring Boot

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-service
  namespace: piclouddoom
spec:
  replicas: 1
  selector:
    matchLabels:
      app: user-service
  template:
    metadata:
      labels:
        app: user-service
    spec:
      containers:
        - name: user-service
          image: docker.io/azizbna/pi-clouddoom-user-service:cv-ollama-fast-v2-
v2
          ports:
            - containerPort: 8080
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: "prod"
            - name: SPRING_DATASOURCE_URL
              valueFrom:
                secretKeyRef:
                  name: user-service-secret
                  key: datasource-url
            - name: SPRING_DATASOURCE_USERNAME
              valueFrom:
                secretKeyRef:
                  name: user-service-secret
                  key: datasource-username
            - name: SPRING_DATASOURCE_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: user-service-secret
                  key: datasource-password
```

10.3 Exemple de Service Kubernetes

```
apiVersion: v1
kind: Service
metadata:
  name: user-service
  namespace: piclouddoom
spec:
  selector:
    app: user-service
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
  type: ClusterIP
```

10.4 Exemple ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: piclouddoom
data:
  KEYCLOAK_REALM: "myapp-realm"
  REDIS_HOST: "redis"
  KAFKA_BOOTSTRAP_SERVERS: "kafka:9092"
```

10.5 Exemple Secret

```
apiVersion: v1
kind: Secret
metadata:
  name: user-service-secret
  namespace: piclouddoom
type: Opaque
stringData:
  datasource-url: "jdbc:postgresql://postgres:5432/userdb"
  datasource-username: "postgres"
  datasource-password: "password"
```

10.6 Exemple PVC

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
  namespace: piclouddoom
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
  storageClassName: local-path
```

10.7 Appliquer les fichiers

```
kubectl apply -f namespace.yaml
kubectl apply -f configmaps/
kubectl apply -f secrets/
kubectl apply -f postgres/
kubectl apply -f redis/
kubectl apply -f kafka/
kubectl apply -f keycloak/
kubectl apply -f services/
kubectl apply -f deployments/
```

ou :

```
kubectl apply -f k8s/
```

si tous les manifests sont dans un seul dossier.

11. Déploiement des composants applicatifs

11.1 Déploiement user-service

Vérification du Deployment :

```
kubectl get deploy user-service -n piclouddoom -o wide
```

Vérification de l'image :

```
kubectl get deploy user-service -n piclouddoom -o  
jsonpath='{.spec.template.spec.containers[0].image}'
```

Vérification des pods :

```
kubectl get pods -n piclouddoom | grep user-service
```

Logs :

```
kubectl logs deployment/user-service -n piclouddoom --tail=100
```

Description :

```
kubectl describe deploy user-service -n piclouddoom
```

11.2 Déploiement community-service

```
kubectl get deploy community-service -n piclouddoom  
kubectl get pods -n piclouddoom | grep community-service  
kubectl logs deployment/community-service -n piclouddoom --tail=100
```

11.3 Déploiement interview-service

```
kubectl get deploy interview-service -n piclouddoom  
kubectl get pods -n piclouddoom | grep interview-service  
kubectl logs deployment/interview-service -n piclouddoom --tail=100
```

11.4 Déploiement training-service

```
kubectl get deploy training-service -n piclouddoom  
kubectl get pods -n piclouddoom | grep training-service  
kubectl logs deployment/training-service -n piclouddoom --tail=100
```

11.5 Vérification globale

```
kubectl get all -n piclouddoom
```

12. Déploiement des composants techniques

12.1 PostgreSQL

Vérifications :

```
kubectl get pods -n piclouddoom | grep postgres  
kubectl get svc -n piclouddoom | grep postgres  
kubectl logs deployment/postgres -n piclouddoom --tail=100
```

Tester la résolution DNS depuis un pod :

```
kubectl run test-dns -n piclouddoom --image=busybox:1.36 --rm -it -- nslookup  
postgres
```

12.2 Redis

```
kubectl get pods -n piclouddoom | grep redis  
kubectl get svc -n piclouddoom | grep redis  
kubectl logs deployment/redis -n piclouddoom --tail=100
```

Tester Redis :

```
kubectl exec -it deployment/redis -n piclouddoom -- redis-cli ping
```

Résultat attendu :

```
PONG
```

12.3 Kafka

```
kubectl get pods -n piclouddoom | grep kafka  
kubectl get svc -n piclouddoom | grep kafka  
kubectl logs deployment/kafka -n piclouddoom --tail=100
```

Points importants :

- vérifier les listeners ;
- vérifier `KAFKA_ADVERTISED_LISTENERS` ;
- vérifier que les microservices utilisent le nom DNS interne du service Kafka ;
- éviter `localhost` dans Kubernetes.

12.4 Keycloak

```
kubectl get pods -n piclouddoom | grep keycloak  
kubectl get svc -n piclouddoom | grep keycloak  
kubectl logs deployment/keycloak -n piclouddoom --tail=100
```

Vérifier l'URL du realm :

```
https://<domain>/realms/myapp-realm
```

ou en interne :

```
http://keycloak:8080/realms/myapp-realm
```

selon la configuration.

13. Configuration Keycloak

13.1 Realm

Realm utilisé :

```
myapp-realm
```

13.2 Client frontend

Client utilisé :

```
angular-client
```

Paramètres importants :

- Valid Redirect URIs ;
- Web Origins ;
- Root URL ;
- Home URL ;
- Access Type ;
- Standard Flow Enabled.

13.3 Rôles

Rôles sécurité :

```
ROLE_USER  
ROLE_ADMIN  
ROLE_MANAGER
```

Important : ces rôles sont des rôles de sécurité. Ils sont différents des statuts métier comme :

```
FREE  
PREMIUM  
ADMIN
```

13.4 Issuer URI côté Spring Boot

Exemple local :

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=http://localhost:8080/  
realms/myapp-realm
```

Exemple Kubernetes interne :

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=http://keycloak:8080/  
realms/myapp-realm
```

Exemple domaine public :

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://<domain>/realms/  
myapp-realm
```

Le choix exact dépend de l'architecture finale. Le point essentiel est que l'issuer dans le token doit correspondre à l'issuer attendu par Spring Boot.

13.5 Providers OAuth

Providers travaillés :

```
Google  
LinkedIn  
GitHub
```

Pour chaque provider, il faut aligner :

- client ID ;
- client secret ;
- callback URL ;
- redirect URI côté provider ;
- redirect URI côté Keycloak ;
- URL publique Cloudflare ;
- URL frontend.

13.6 Diagnostic OAuth

En cas d'erreur OAuth :

1. lire l'erreur affichée dans le navigateur ;
2. vérifier la callback URL dans le provider ;
3. vérifier l'Identity Provider dans Keycloak ;
4. vérifier les logs Keycloak ;
5. vérifier le protocole HTTP/HTTPS ;
6. vérifier le domaine ;
7. vérifier le slash final ;
8. refaire un test en navigation privée.

Commandes utiles :

```
kubect1 logs deployment/keycloak -n piclouddoom --tail=200
```

14. Exposition publique avec Cloudflare

14.1 Objectif

Cloudflare a été utilisé pour rendre l'application accessible via un domaine public et faciliter l'utilisation de HTTPS/DNS.

14.2 Éléments à vérifier

- DNS record ;
- proxy activé ou désactivé selon besoin ;
- domaine pointant vers l'origine ;
- tunnel si utilisé ;
- service Kubernetes exposé ;
- frontend accessible ;
- Keycloak accessible ;
- OAuth redirect URIs alignées.

14.3 Erreur Cloudflare 1033

Erreur rencontrée après reboot :

Cloudflare Error 1033

Causes possibles :

- tunnel non démarré ;
- origine indisponible ;
- service Kubernetes arrêté ;
- ingress non accessible ;
- DNS incorrect ;
- cluster non prêt après reboot.

Diagnostic :

```
kubectl get nodes
kubectl get pods -n picloudboom
kubectl get svc -n picloudboom
```

Puis vérifier le service/tunnel Cloudflare selon la méthode utilisée.

14.4 Pourquoi Cloudflare est important pour OAuth

Les providers OAuth comme Google, LinkedIn et GitHub exigent des URLs publiques exactes.

Une erreur de domaine ou de protocole peut casser le login.

Exemples de différences qui provoquent une erreur :

```
http au lieu de https  
ancien domaine  
mauvais port  
slash final différent  
callback Keycloak incorrect
```

15. IA et parsing de CV

15.1 Objectif fonctionnel

Lorsqu'un utilisateur upload un CV PDF, le système doit :

1. stocker le CV ;
2. permettre la consultation du CV ;
3. extraire le texte ;
4. envoyer le texte à un modèle IA ;
5. recevoir une réponse JSON ;
6. enrichir automatiquement le profil utilisateur.

15.2 Stockage du CV

Dossier logique :

```
uploads/cvs
```

URL relative typique :

```
cvs/user-<uuid>-cv-<uuid>.pdf
```

15.3 Données extraites

Champs visés :

```
bio
skills
educations
experiences
projects
certifications
```

15.4 Structure JSON attendue

```
{
  "bio": "...",
  "skills": ["Java", "Spring Boot", "Docker"],
  "educations": [
    {
      "degree": "...",
      "institution": "...",
      "startDate": "...",
      "endDate": "..."
    }
  ],
  "experiences": [
    {
      "title": "...",
      "company": "...",
      "startDate": "...",
      "endDate": "...",
      "description": "..."
    }
  ]
}
```

15.5 Passage Gemini vers Ollama

Stratégie initiale : Gemini.

Problèmes :

- clé API ;
- quota ;
- URL d'appel ;
- dépendance externe ;
- coût potentiel.

Stratégie adoptée : Ollama.

Modèle étudié/utilisé :

```
llama3.2:3b
```

15.6 Tester Ollama localement

```
ollama list
ollama pull llama3.2:3b
ollama run llama3.2:3b
```

Avec API :

```
curl http://localhost:11434/api/tags
```

Exemple d'appel :

```
curl http://localhost:11434/api/generate
-H "Content-Type: application/json"
-d '{
  "model": "llama3.2:3b",
  "prompt": "Extract skills from this CV text and return JSON only: Java
Docker Kubernetes",
  "stream": false
}'
```

15.7 Déploiement IA hybride

L'IA peut être lourde pour OpenStack. Pour cette raison, une stratégie hybride a été étudiée :

```
OpenStack/Kubernetes : application principale
Azure Container Apps : Ollama / service IA
Cloudflare : exposition publique
```

Configuration Azure envisagée :

```
min replicas = 0
max replicas = 1
```

Avantage : réduire le coût.

Limite : cold start au premier appel.

Point important : si le modèle n'est pas préchargé dans l'image ou stocké dans un volume persistant, il peut être retéléchargé au démarrage.

16. Monitoring et observabilité

16.1 Monitoring de base avec kubectl

Commandes principales :

```
kubectl get nodes
kubectl get pods -n piclouddoom
kubectl get deploy -n piclouddoom
kubectl get svc -n piclouddoom
kubectl describe pod <pod-name> -n piclouddoom
kubectl logs <pod-name> -n piclouddoom
kubectl logs deployment/<deployment-name> -n piclouddoom --tail=100
kubectl rollout status deployment/<deployment-name> -n piclouddoom
```

16.2 Éléments surveillés

- état des pods ;
- redémarrages ;
- consommation CPU/RAM ;
- erreurs applicatives ;
- erreurs OAuth ;
- erreurs de connexion DB ;
- disponibilité Keycloak ;
- disponibilité PostgreSQL ;
- disponibilité frontend ;
- état des services Kubernetes.

16.3 Spring Boot Actuator

Endpoints utiles :

```
/actuator/health
/actuator/info
/actuator/metrics
```

Test :

```
curl http://<service-url>/actuator/health
```

16.4 Monitoring avancé recommandé

Stack proposée :

```
Prometheus  
Grafana  
Alertmanager  
node-exporter  
kube-state-metrics
```

Objectif :

- collecter les métriques ;
- visualiser l'état du cluster ;
- détecter les anomalies ;
- déclencher des alertes.

17. Détection d'attaques et anomalies

17.1 Objectif

La détection d'attaques vise à identifier des comportements anormaux ou suspects.

17.2 Exemples d'anomalies

```
Trop de requêtes en peu de temps  
Trop d'erreurs 401/403  
Tentatives répétées de login  
Accès à des endpoints sensibles  
Pic de CPU/RAM  
Redémarrages fréquents de pods  
Scan d'URLs  
Erreurs répétées Keycloak
```

17.3 Sources de données

```
Logs des microservices  
Logs Keycloak  
Logs Ingress / proxy  
Métriques Kubernetes  
Métriques Prometheus  
Événements Kubernetes  
Codes HTTP
```

17.4 Exemple de règles simples

```
Si une IP génère plus de 100 requêtes en 1 minute -> suspect  
Si un utilisateur génère plus de 10 erreurs 401 en 5 minutes -> alerte  
Si un pod redémarre plus de 3 fois en 10 minutes -> instabilité  
Si un endpoint admin est appelé sans rôle admin -> événement critique
```

17.5 Valeur ajoutée

Cette partie montre que le projet n'est pas seulement déployé. Il est aussi pensé pour l'exploitation, la sécurité et la surveillance.

18. Rollout, rollback et correction des erreurs

18.1 Vérifier l'état d'un rollout

```
kubectl rollout status deployment/user-service -n picloudoom --timeout=180s
```

18.2 Faire un rollback

```
kubectl rollout undo deployment/user-service -n picloudoom
```

18.3 Voir l'historique

```
kubectl rollout history deployment/user-service -n picloudoom
```

18.4 Modifier l'image d'un Deployment

```
kubectl set image deployment/user-service user-service=docker.io/azizbna/pi-clouddoom-user-service:<new-tag> -n piclouddoom
```

18.5 Redémarrer un Deployment

```
kubectl rollout restart deployment/user-service -n piclouddoom
```

19. Problèmes rencontrés et diagnostic

19.1 ErrImagePull

Symptôme :

```
user-service-xxxxx 0/1 ErrImagePull
```

Diagnostics :

```
kubectl describe pod <pod-name> -n piclouddoom
kubectl get deploy user-service -n piclouddoom -o
jsonpath='{.spec.template.spec.containers[0].image}'
```

Causes possibles :

- tag inexistant ;
- image non poussée ;
- image privée ;
- mauvais nom d'image ;
- problème Docker Hub ;
- problème réseau.

Correction :

- vérifier Docker Hub ;
- corriger le tag ;
- republier l'image ;
- créer un imagePullSecret si image privée ;
- relancer le Deployment.

19.2 Pod en 0/1 Ready

Diagnostic :

```
kubectl describe pod <pod-name> -n piclouddoom  
kubectl logs <pod-name> -n piclouddoom
```

Causes :

- application crash ;
- variable manquante ;
- base inaccessible ;
- Keycloak inaccessible ;
- erreur de port ;
- readiness probe incorrecte.

19.3 CrashLoopBackOff

Diagnostic :

```
kubectl logs <pod-name> -n piclouddoom --previous  
kubectl describe pod <pod-name> -n piclouddoom
```

Causes :

- exception au démarrage ;
- mauvais profil Spring ;
- datasource incorrecte ;
- dépendance absente ;
- mémoire insuffisante.

19.4 OAuth redirect error

Diagnostic :

- lire erreur navigateur ;
- vérifier Keycloak ;
- vérifier provider OAuth ;
- vérifier Cloudflare ;
- vérifier URL publique ;
- vérifier logs Keycloak.

Commande :

```
kubectl logs deployment/keycloak -n piclouddoom --tail=200
```

19.5 Cloudflare 1033

Diagnostic :

```
kubectl get nodes  
kubectl get pods -n piclouddoom  
kubectl get svc -n piclouddoom
```

Puis vérifier le tunnel ou le DNS Cloudflare.

19.6 Problème StorageClass

Diagnostic :

```
kubectl get storageclass
```

Correction :

```
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-  
provisioner/master/deploy/local-path-storage.yaml
```

19.7 SSH host key changed

Correction :

```
ssh-keygen -f ~/.ssh/known_hosts -R <IP>
```

20. Commandes de validation finale avant jury

20.1 Vérifier cluster

```
kubectl get nodes -o wide
```

20.2 Vérifier namespace

```
kubectl get all -n piclouddoom
```

20.3 Vérifier pods

```
kubectl get pods -n piclouddoom -o wide
```

20.4 Vérifier services

```
kubectl get svc -n piclouddoom
```

20.5 Vérifier deployments

```
kubectl get deploy -n piclouddoom
```

20.6 Vérifier image user-service

```
kubectl get deploy user-service -n piclouddoom -o  
jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

20.7 Vérifier logs user-service

```
kubectl logs deployment/user-service -n piclouddoom --tail=100
```

20.8 Vérifier logs Keycloak

```
kubectl logs deployment/keycloak -n piclouddoom --tail=100
```

20.9 Vérifier StorageClass

```
kubectl get storageclass
```

20.10 Vérifier PVC

```
kubectl get pvc -n piclouddoom
```

20.11 Vérifier événements récents

```
kubectl get events -n piclouddoom --sort-by=.lastTimestamp
```

21. Tests applicatifs à préparer

21.1 Test accès frontend

Action : ouvrir le domaine public de l'application.

Résultat attendu : l'interface Angular s'affiche.

21.2 Test login classique

Action : se connecter avec un utilisateur existant.

Résultat attendu : redirection vers le profil ou dashboard.

21.3 Test OAuth LinkedIn / Google / GitHub

Action : cliquer sur le bouton OAuth.

Résultat attendu : redirection vers le provider puis retour vers l'application.

21.4 Test profil utilisateur

Action : afficher le profil.

Résultat attendu : les informations utilisateur sont chargées depuis le backend.

21.5 Test upload CV

Action : uploader un CV PDF.

Résultat attendu :

- le fichier est accepté ;
- l'URL du CV est sauvegardée ;
- le bouton View CV fonctionne ;
- les données extraites apparaissent si l'IA est active.

21.6 Test View CV

Action : cliquer sur View CV.

Résultat attendu : le PDF s'ouvre ou se télécharge correctement.

21.7 Test parsing IA

Action : uploader un CV contenant des compétences visibles.

Résultat attendu : le profil affiche des compétences comme :

```
Java  
Spring Boot  
Docker  
Kubernetes  
Angular
```

21.8 Test logs pendant une action

Pendant une action frontend, lancer :

```
kubectl logs deployment/user-service -n picloudoom -f
```

Puis montrer au jury que l'action génère des logs backend.

22. Fichier de preuves à capturer en screenshots

Avant la validation, préparer des captures :

```
1. kubectl get nodes  
2. kubectl get pods -n picloudoom  
3. kubectl get svc -n picloudoom  
4. kubectl get deploy -n picloudoom
```

5. image Docker Hub du user-service
6. logs user-service Running
7. frontend accessible
8. login Keycloak
9. providers OAuth Keycloak
10. upload CV
11. View CV
12. monitoring ou commandes de diagnostic
13. Cloudflare DNS/tunnel si utilisé
14. Docker Hub repository
15. GitHub repository / branche finale

Ces captures peuvent être ajoutées à un rapport final ou utilisées si la connexion réseau tombe pendant la soutenance.

23. Checklist finale pour demain matin

23.1 Infrastructure

- ☐ OpenStack accessible
- ☐ VMs allumées
- ☐ SSH fonctionne
- ☐ cluster Kubernetes accessible
- ☐ nodes Ready

23.2 Kubernetes

- ☐ namespace `picclouddoom` existe
- ☐ pods Running
- ☐ deployments Available
- ☐ services présents
- ☐ PVC Bound
- ☐ StorageClass disponible

23.3 Application

- ☐ frontend accessible
- ☐ backend répond
- ☐ user-service Running
- ☐ community-service Running
- ☐ interview-service Running
- ☐ training-service Running
- ☐ PostgreSQL Running
- ☐ Redis Running

- [] Kafka Running ou expliqué si non requis pendant demo
- [] Keycloak Running

23.4 Authentication

- [] login fonctionne
- [] Keycloak accessible
- [] realm correct
- [] client frontend correct
- [] OAuth providers vérifiés
- [] redirect URIs correctes

23.5 IA

- [] upload CV fonctionne
- [] View CV fonctionne
- [] parsing IA fonctionne ou fallback expliqué
- [] Ollama/Azure stratégie prête à expliquer

23.6 Monitoring / diagnostic

- [] commandes kubectl prêtes
- [] logs consultables
- [] events consultables
- [] scénario de panne prêt
- [] captures backup prêtes

24. Plan de secours si quelque chose ne fonctionne pas pendant la validation

24.1 Si le frontend ne s'ouvre pas

Montrer :

```
kubectl get pods -n picloudboom  
kubectl get svc -n picloudboom
```

Puis expliquer que l'application est déployée mais que l'accès public dépend de Cloudflare/tunnel/ingress.

Utiliser les screenshots préparés.

24.2 Si un pod est en erreur

Montrer la démarche de diagnostic :

```
kubectl describe pod <pod-name> -n piclouddoom  
kubectl logs <pod-name> -n piclouddoom  
kubectl get events -n piclouddoom --sort-by=.lastTimestamp
```

Le jury valorise souvent la capacité à diagnostiquer plus que le fait que tout soit parfait.

24.3 Si OAuth échoue

Expliquer que l'OAuth dépend de l'alignement exact entre :

```
frontend  
Keycloak  
provider OAuth  
Cloudflare  
domaine public
```

Puis montrer la configuration Keycloak et les redirect URIs.

24.4 Si Ollama/IA est lent

Expliquer que le modèle IA consomme beaucoup de ressources et que c'est pour cette raison qu'une stratégie hybride avec Azure a été proposée.

Dire :

L'IA n'est pas appelée en permanence. Elle est utilisée au moment de l'upload CV. C'est donc un bon candidat pour un service séparé, scalable à la demande.

25. Résumé technique final

Pendant la phase de déploiement, nous avons :

- préparé l'infrastructure OpenStack ;
- créé un cluster Kubernetes ;
- organisé les ressources dans le namespace `piclouddoom` ;
- conteneurisé les microservices avec Docker ;
- publié les images sur Docker Hub ;

- déployé les services dans Kubernetes ;
- configuré PostgreSQL, Redis, Kafka et Keycloak ;
- corrigé les erreurs d'image et de pods ;
- utilisé rollout et rollback ;
- exposé l'application avec Cloudflare ;
- configuré OAuth Google, LinkedIn et GitHub ;
- intégré une fonctionnalité IA basée sur le parsing de CV ;
- étudié Ollama et Azure pour la stratégie IA ;
- préparé le monitoring ;
- défini une stratégie de détection d'anomalies ;
- documenté les commandes et procédures de diagnostic.

Ce dossier prouve que le projet a été traité comme une application cloud-native complète, avec une vraie démarche DevOps : build, containerisation, orchestration, exposition, sécurité, observabilité, diagnostic et amélioration continue.